Cybersecurity

RESEARCH

Open Access

# Understanding and boosting fully homomorphic encryption applications on GPU

Shengyu Fan[1,2], Xianglong Deng[1,2], Xulong Tang[3], Weizhi Xu[4] and Mingzhe Zhang[4*]

**Abstract**

Fully Homomorphic Encryption (FHE) is considered one of the most promising candidates for future privacy computing since it allows to directly compute the encrypted data. Though FHE enables secure computation on untrusted servers, its utilization is limited due to the dramatically increased computation workload and a 4–5 orders of magnitude slowdown ratio. Several previous works have been proposed to accelerate FHE on GPUs, while most of these efforts focus on the algorithm or scheduling and still leave a significant performance gap. However, there is a lack of understanding of the FHE applications from the micro-architecture level, which is important for further optimization of FHE applications or designing hardware accelerators. In this paper, we make a detailed analysis for running FHE on GPUs and present the following key performance bottlenecks at the micro-architecture level: (1) FHE applications require more capacity for the I-cache than other workloads; (2) FHE causes large amount of pipeline stalls due to the Read-After-Write (RAW) issues and significantly hurts the performance due to poor hardware utilization; (3) the capacity of texture cache is severely under-utilized in FHE execution. We propose a simple yet effective pure-hardware scheme for boosting FHE on GPUs based on these observations. Our proposed scheme significantly reduces the RAW-caused pipeline stalls by adding a small forwarding buffer. Besides, our proposed scheme also leverages a partition of the texture cache as the victim buffer for the proposed forwarding buffer to minimize the hardware overheads. We explore various design choices to balance the performance and hardware complexity. The experiment results show that our design improves the performance of the end-to-end FHE workflow by 47.5% with only 0.5% additional hardware overhead.

**Keywords** GPU, Fully homomorphic encryption, Architecture

## Introduction

Fully Homomorphic Encryption (FHE) is proposed as an ideal solution for privacy protection in cloud computing, which allows direct computation on the encrypted data and avoids exposing the data privacy to the untrusted environment. As shown in Fig. 1, the user encrypts the sensitive data *m* at client and then uploads the encrypted data (*Encrypt(m)*) to the untrusted server. The encrypted data is directly processed by the application using the specific FHE operators on the server side. Then, the user downloads the result (*f(Encrypt(m))* and decrypts it at the client side to get the required result (*f(x)*). In the whole procedure, the data is secure, and the attackers cannot learn anything about it.

Due to the above feature, FHE is considered to be a good fit for several popular applications requiring data privacy, such as personalized machine learning inference (Dathathri et al. 2020) and training (Han et al. 2019). Unfortunately, FHE execution suffers high performance overheads that prevent its wide adoption. One reason is that the size of the encrypted data increases by 2 to 3

*Correspondence:
Mingzhe Zhang
smartzmz@gmail.com
[1] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3] University of Pittsburgh, Pittsburgh, USA
[4] Ant Group, Beijing, China

Springer Open

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

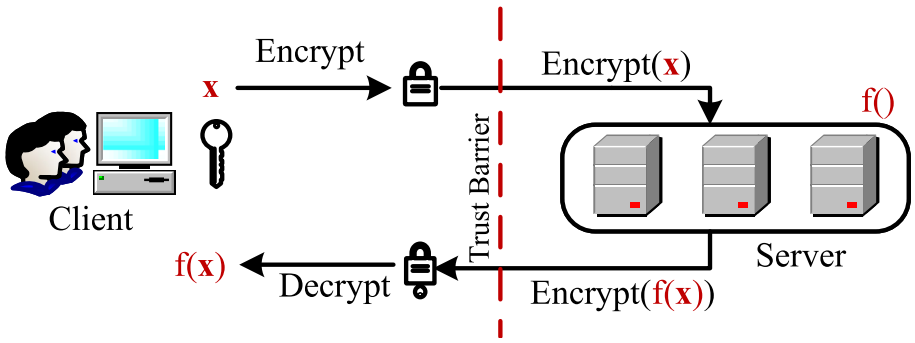



**Fig. 1** The conceptual example of how Fully Homomorphic Encryption guarantees the data privacy

orders of magnitude, which leads to much more computation workload. In the early FHE implementation, the computation is about $10^9$ times slower than performing computation using unencrypted data (Gentry 2009). Although various optimizations and new implementations have been proposed, the performance overhead of FHE is still significant, which is 4 to 5 orders of magnitude for the slowdown ratio (Dathathri et al. 2019, 2020).

Since the high-end GPUs are widely equipped in the cloud servers, it is natural to accelerate the computation and data-intensive FHE application with GPU. Prior efforts have been carried out to take advantage of the massive parallelism in GPUs (Boemer et al. 2021; Morshed et al. 2020). While these efforts achieve remarkable performance improvements with algorithm-level (Wang et al. 2012) and runtime scheduling optimizations (Jung et al. 2021; Kim et al. 2020), it is still difficult to bridge the performance gap for the real workloads. For example, though FHE gets performance improvement on GPUs compared to CPUs, one FHE multiply still requires 7.96 ms (Jung et al. 2021) or 4.6 ms (Fan et al. 2025). The fundamental reason behind this is that the modern GPU architecture is weak in handling FHE execution characteristics. Therefore, a micro-architectural understanding for FHE running on GPUs is necessary before further improvement.

In this paper, we present the first detailed analysis of running FHE on GPUs. We observe that the main reason behind the poor performance is the significant amount of pipeline stalls that consequently lead to hardware under-utilization. Among the various reasons behind the pipeline stall, the *L1 I-Cache miss* and the *Read-After-Write (RAW)* dependency issues are the two most significant reasons. While the I-cache miss rate can be improved by enabling I-cache prefetching, existing techniques are ineffective in addressing the RAW issue of the FHE execution on the GPU due to the high design overhead and the low hardware efficiency. Therefore, we propose a pure hardware solution, which is named *Global Forwarding Table (GFT)* in each Streaming Multiprocessor (SM) to reduce the pipeline stall caused by the RAW issue. *GFT* provides a shared buffer for all warps in the same SM to buffer the value before its writeback is finished. In this way, the following instructions can load the required operands from GFT and avoid the RAW stall. Besides, to reduce the hardware overhead of the proposed *GFT*, we also propose to utilize the idle Texture-Cache (T-Cache) as a victim buffer for the *GFT* spilling. The experimental results show that, by using *GFT* together with L1 I-Cache prefetching, the overall performance of the FHE workflow can achieve 1.9× speedup with only 0.5% of hardware overhead increment.

The contributions of this paper are summarized as follows:

- We provide the first detailed analysis at the micro-architecture level for running FHE on GPU. We observe that the main reason for the low performance is the GPU under-utilization caused by large number of pipeline stalls.
- We analyze the reasons for the pipeline stalls and quantitatively evaluate the effectiveness of the state-of-the-art techniques addressing the stalls. We find that though prefetching can reduce the pipeline stall caused by L1 I-Cache miss, the RAW issue is more serious.
- We propose a pure hardware forwarding technique with minor hardware overhead and integrate it with the state-of-the-art GPU architecture.
- We evaluate our proposed technique by simulating a GPU integrated with proposed MemoPad and running a typical FHE workflow on it, the results show that MemoPad improves the performance of the whole workflow by 47.5% with the minor hardware overhead (0.5%).

## Background

### Threat model

In this work, the users upload their data to the cloud servers and fulfil the data-intensive computation tasks remotely. The cloud servers are equipped with the high-end processors and GPUs, which are vulnerable to attacks and may cause privacy data leakage. With emerging technologies, adding specially designed components allows the data to remain encrypted until it enters the CPUs and the GPUs, therefore, reducing the need for protection. However, such designs still can not completely avoid the privacy data leakage, due to the attacks during the non-encrypted data computation, such as physically monitoring the signals in the CPU and GPU cards. Therefore, in our threat model, we still assume the cloud computing environment is outside the trusted boundary.

### GPU architecture

Figure 2 illustrates our targeted baseline GPU architecture. Specifically, the GPU comprises multiple Streaming Multiprocessors (SMs), and the SMs are connected to memory partitions through an interconnection. Each SM is equipped with a private L1 data cache (D-cache), L1 instruction cache (I-cache), texture cache, and scratchpad memory. The computation executed on the GPU is called kernel. A kernel consists of multiple thread blocks (TBs). A TB further consists of multiple warps, which usually comprise 32 threads executing in a lockstep fashion. The warp schedulers in the SM choose ready-to-execute warps and perform lightweight context switching to hide long latency operations and stalls. We use the GTO warp scheduler in the baseline GPU (Rogers et al. 2012).

The execution in each SM follows a 5-stage in-order pipeline (i.e., fetch, decode, issue, read operands, and execution/write back). In fetch, the instruction is read from the I-cache and sent to the decode unit. The decode unit stores the decoded instructions in the I-buffer. In the issue stage, the warp scheduler chooses available warps to execute the instruction from I-buffer. The SIMT stack is used to track and handle the intra-warp control divergences. Also, the scoreboard in the issue stage is used to track the dependencies among the instructions. If a dependency is observed, the warp scheduler switches to other warps. Once the instruction is issued, it reads the operands from the register file and then executes it on the SIMD cores. The results are finally written back to the memory or the register file.

It is important to note that, hardware forwarding from the execution stage to the read operands stage for dependent instructions is not supported in modern GPU execution. The main reason behind this is that forwarding vector data involves much higher hardware design overheads and latency overheads than forwarding scalar data in CPU execution (Gilani et al. 2013). To double confirm, we also conduct microbenchmarking on modern GPUs and the results indicate there is no pipeline forwarding in GPU execution.

### Basics of homomorphic encryption

Fully Homomorphic Encryption supports direct computation on the encrypted data with the specific implemented operations. Users can get the required result by decrypting the results. Since the first practical implementation of FHE was proposed in 2009 (Gentry 2009), a series of FHE schemes have been proposed for improving the capability or the performance, such as BGV (Yagisawa 2015), BFV (Fan and Vercauteren 2012), ATV-FHE (Doroz et al. 2014) and CKKS (Cheon et al. 2017). Although different schemes use various encryption algorithms, the procedures of these FHE schemes

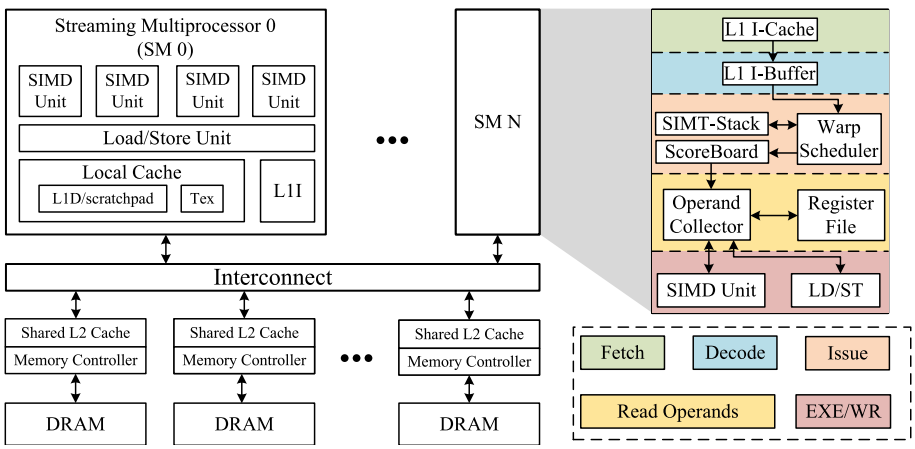



**Fig. 2** Baseline GPU architecture

are similar. In this subsection, we briefly introduce how FHE works and the way of mapping an FHE workload to GPU. To simplify the description, here we take the ATV-FHE (also referred to as DHS) (Doroz et al. 2014) as an example.

#### *FHE overview*

As shown in Fig. 3a, the whole procedure of FHE includes several key steps. First, the input raw data is encoded as the polynomial (*Plaintext*). Then, the plaintext is encrypted to ciphertext by using the public key, which is still of the polynomial type. After that, a series of ciphertexts are involved in the calculation using the specific operators (usually called *evaluation*) to provide the ciphertext result. Finally, the result is decrypted with the private key and then decoded to the results in raw data. In such a procedure, the specific schemes differ in the way of encoding/decoding and encryption/decryption. However, for all schemes, the *Encryption*, the *Evaluation*, and the *Decryption* (marked with the red box in Fig. 3a) are computation-intensive and usually considered as the workload fit for GPU acceleration. This paper is also based on this intuition, and the *Encode* and *Decode* are not in the scope. Note that, although previous works show that the *Evaluation* will become cheaper if one of the operands is made unencrypted (Brutzkus et al. 2019), we do not consider these optimizations that trade privacy for performance. To keep the generality, here we only consider the situation where all operands are encrypted.

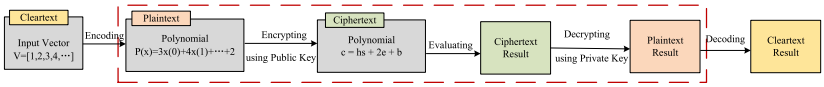
#### *Workflow of the key FHE operations*

When we take a closer look at the key FHE operations (i.e., *Encryption*, *Evaluation*, and *Decryption*), the high level workflow for these operations are very similar. As shown in Fig. 3b, the input polynomial ($P$) is firstly converted to a group of polynomials ($p_1, p_2, \ldots, p_n$). Each coefficient in $p_n$, marked as $c_{ni}$, is a segment of data generated from the $D_i$ (i.e., one of the coefficients for $P$) by the kernel *CRT*. After that, all of the generated polynomials ($p_n$) are successively processed by a series of *NTT* kernels (i.e., $\text{NTT}_1$, $\text{NTT}_2$, $\text{NTT}_3$) separately and then the results are sent to the `Computation Operation` kernel together with the other polynomial group. Finally, the output polynomial group of the `Computation Operation` is processed by the *INTT* kernels (i.e., $\text{INTT}_1$, $\text{INTT}_2$, $\text{INTT}_3$) and the output is converted to the single result polynomial by the *ICRT* kernel. Note that, for different operations, the `Computation Operation` indicates various operations as summarized in Table 1. Besides, we also list all related kernels of ATV-FHE workflow in Table 2. In this paper, we use these kernels as the benchmarks.

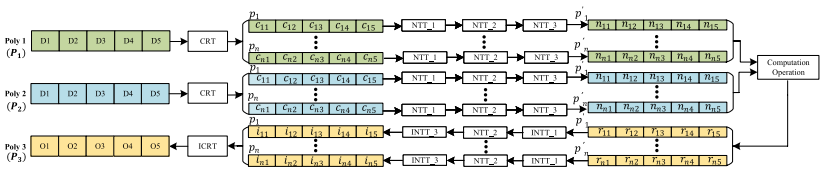
#### *Mapping FHE workload to GPU*

According to the above discussion, it is natural to parallelize and map the workload to GPU. For DHS, there are two mapping methods:

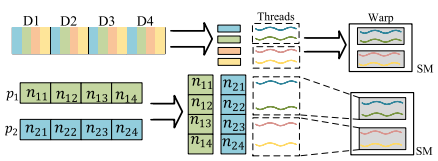
- As shown in Fig. 3b, for some kernels (i.e., CRT), the input is an individual polynomial (i.e., $P_1$, $P_2$) and the



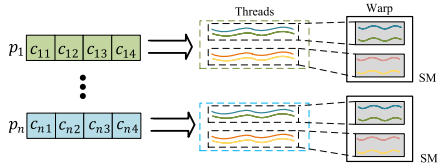

(a) A high-level view of the FHE procedure.




(b) The expanded workflow of the operations based on the polynomials. Taking ATV-FHE (also referred as DHS) [**?** ] as an example.




(c) Mapping the threads for the CRT-related and `Computation Operation` kernels to GPU.




(d) Mapping the threads for the NTT-related kernels to GPU.

**Fig. 3** An example of FHE workflow

**Table 1** Detailed description for the *Computation Operation* in Fig. 3b

| HE stage | Poly 1 | Poly 2 | Operation |
|---|---|---|---|
| Encryption | Public Key | Error | $T = P_1 \odot P_2$ |
| | | Distribution | $O = \mathbf{Br}(T)$ |
| HMultiply | Ciphertext 1 | Ciphertext 2 | $T = P_1 \odot P_2$ |
| | | | $O = \mathbf{Br}(T)$ |
| HADD | Ciphertext 1 | Ciphertext 2 | $T = P_1 \oplus P_2$ |
| Decryption | Private Key | Ciphertext | $T = P_1 \odot P_2$ |
| | | Result | $O = \mathbf{Br}(T)$ |

$\odot$ indicates the Dot Product Operation; ($\mathbf{Br}()$) indicates the Barrett Reduction Operation; $\oplus$ indicates the Add Operation

**Table 2** ATV-FHE Kernels

| Kernel | Description |
|---|---|
| ntt_1 | Performing the column FFT operation |
| ntt_2 | Performing the twiddle factor's multiplication |
| ntt_3 | Performing the row FFT operation |
| intt_1 | Performing the column Inverse-FFT operation |
| intt_2 | Performing the row Inverse-FFT operation |
| crt | Transforming big integer coefficients to residue number |
| icrt | Transforming the residue number to big integer coefficients |
| barrett_*n* | Performing a customized Barrett reduction operation |
| ntt_mul | Performing Element-wise modular multiplication |
| ntt_add | Performing Element-wise modular addition |

output is a group of polynomials (i.e., $p_1$,..., $p_n$). For this kind of kernels, the input data for one thread is from the different coefficients of one polynomial. For instance, as shown in the top of Fig. 3c, when a polynomial is input to the CRT kernel, each thread is allocated a group of segments as the input and each segment is from an individual coefficient (i.e., $D_1$, $D_2$ ,..., $D_4$) of the input polynomial. On the other hand, for the ICRT and the evaluation-related kernels, the input is a group of polynomials and the output is one or a group of polynomials (shown in Fig. 3b). For such situation, as shown in the bottom of the Fig. 3c, the input for each thread is a group of coefficients (i.e., ($n_{11}$, $n_{21}$), ($n_{12}$, $n_{22}$), etc.) and each coefficient is from a different input polynomial (i.e., $p_1$, $p_2$). Finally, the threads are organized as warps and mapped to the GPU.

- As shown in Fig. 3b, for the NTT-related kernels, each kernel takes an individual polynomial as the input (i.e., $p_1$,..., $p_n$) and output a new polynomial (i.e., $p'_1$,..., $p'_n$). In such situation, the coefficients from the same polynomial are allocated to one or more threads from the kernel (as shown in Fig. 3d). After that, the threads are mapped to the GPU as warps.

Note that, since the FHE schemes usually only differ in the encryption and decryption algorithms, the above parallelization and mapping method can be used in various FHE schemes.

## Micro-architecture level analysis for running FHE on GPU

In this section, we provide a comprehensive understanding of running FHE on GPU at the micro-architecture level. We first show that the performance degradation is mainly caused by a large amount of pipeline stalls. Then we conduct a detailed analysis and root the most significant two sources of the pipeline stalls. Finally, we explore the impact of the texture-cache capacity and summarize our opportunity for a pure hardware solution to accelerate the FHE on GPU.

### What cause the low performance?

As we introduced in "GPU architecture" section, GPU utilizes a large amount of SIMD cores with the static pipeline. Once the pipeline is stalled, the scheduler directly switches to other pending threads to fully utilize the hardware resources. Therefore, high pipeline utilization generally yields good execution performance in GPUs.

Based on this execution paradigm, we investigate the SM occupancy of running FHE workflow (see "Methodology" section) on NVIDIA 1080Ti and present the results in Fig. 4. Surprisingly, though with the algorithm optimization, all kernels suffer low SM occupancy. The average SM occupancy of the NTT-related kernels is only 3.6%, while the highest occupancy is only 25.8% in *barrett_1*.

To explore the reasons for the low hardware utilization, we execute the FHE workflow in a simulated GPU environment (see "Backup forwarding buffer" section for detailed configurations) and analyze the pipeline stall time breakdown for each kernel and the whole workflow. Besides, we also investigate the GPU pipeline stall for non-FHE applications from Rodinia Benchmark Suite (Che et al. 2009). Note that, since the GPU switches to the other warp when a pipeline stall occurs, here we only take into account the stall cycles that cannot be hidden. The results are shown in Fig. 5 and we can make the following observations:

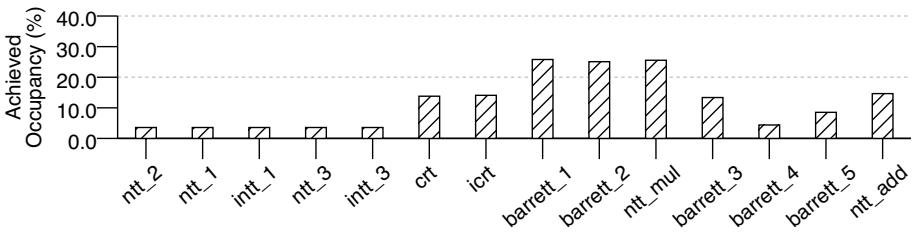
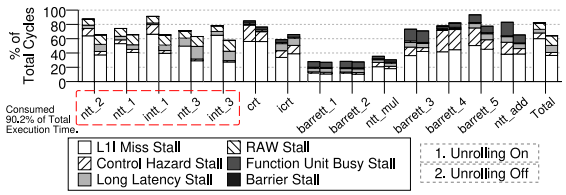
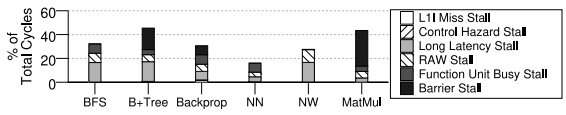



**Fig. 4** The SM occupancy of running FHE workflow on NVIDIA 1080Ti




(a) The breakdown of pipeline stalls when running FHE kernels. For each kernel, the first bar represents the data for `unrolling on`, while the second bar represents the data for `unrolling off`.




(b) The breakdown of pipeline stall for non-FHE applications.

**Fig. 5** Quantitative analysis for GPU pipeline stall

- Only a few of the kernels consume most of the execution time (about 90.2%, as marked with red box). However, during the execution of these kernels, the portion of pipeline stall time is 80.4% as a geometric mean, and is up to 91.1% in `intt_1`.
- Surprisingly, the most significant reason for pipeline stall is the *L1 I-Cache miss*. In particular, the geometric mean of the L1 I-Cache Miss stall is 59.5% for the five most time consuming kernels.
- Besides, the *Read-After-Write (RAW)* issues also cause large amount pipeline stalls, which is about 8.8% of the total execution time. In particular, for the five most time-consuming kernels, *RAW* causes 9.2% of the pipeline stall as a geometric mean.
- Differential with the expectation, the bandwidth has limited impact on the performance. As shown in Fig. 5a, as a geometric mean, only 5.1% the pipeline stall time is caused by the long memory access latency (up to 11.2% for `ntt_3`). This is because the GPU uses TLP to hide the memory access latency. Therefore, we can infer that the bandwidth increment has less impact on the performance improvement.
- Interestingly, the pipeline stalls in the non-FHE applications are much fewer. As shown in Fig. 5b, the proportion of pipeline stall cycles is lower than 50% in all applications. Also, the main reason for the stalls is long latency for memory accesses, rather than the L1I cache miss or RAW issues.

Besides, we also explore the impact of Thread Level Parallelism (TLP) on the pipeline stall. As shown in Fig. 6, when increasing the threads allocated in one SM, the pipeline stall time of most kernels obtains minor decrement. However, the amount of the stall time is still high and the fundamental reasons are remained. Therefore, we can infer that the TLP has limited impact on the FHE acceleration.

Ideally, we want a GPU running FHE with less pipeline stalls. Therefore, the key to boosting FHE on GPU is to reduce the *L1 I-Cache misses* and the *RAW* issues.

## Are the things getting worse?

### *L1 I-Cache miss*

We evaluate two techniques for reducing L1 I-Cache misses. The first technique is the loop unrolling (Murthy et al. 2010). Interestingly, when we turn off the unrolling, the pipeline stall time caused by L1 I-Cache misses is reduced for the entire workflow and almost all kernels

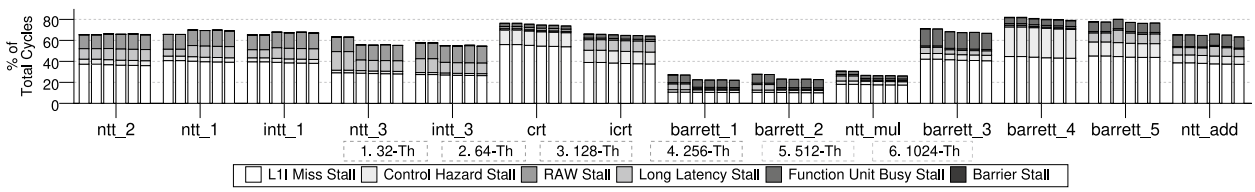



**Fig. 6** The impact of TLP on the pipeline stall of the FHE kernels. The *n-Th* refers allocating *n* threads in one SM

(as shown in Fig. 5a). This indicates that the L1 I-Cache misses are mainly caused by the capacity limitation, and the instruction increment caused by unrolling technique aggravates this issue. As shown in Fig. 7, when the unrolling is turned off, the L1 I-Cache miss rate reduces from 12.4% to 5.3%. This also proves the negative impact of unrolling technique on the FHE performance. Furthermore, we explore the impact of prefetching technique on L1 I-Cache misses. We simulate the prefetching technique as introduced in Smith (1982) to the L1 I-Cache. Besides, we also add an active prefetching scheme which issues extra prefetch requests when there is free space in the L1 I-Cache. The whole prefetching scheme is as shown in Fig. 8. From the Fig. 9, we can conclude that the prefetching reduces the pipeline stall caused by L1 I-Cache misses by 21.4%.

#### *Read-after-write dependency issue*

Although turning off unrolling and applying prefetching on L1 I-Cache helps to reduce the L1 I-Cache misses, they will probably exacerbate the *RAW* issues, which is the other main reason for pipeline stall. For example, as shown in Fig. 9 when turning off the unrolling technique, the pipeline stall caused by *RAW* increases from 8.8% to 13.5%. On the other hand, when applying L1 I-Cache prefetching, the *RAW* issues increase by 13.9% for the total workflow. As shown in Fig. 5, 8.8% of the total execution time is occupied by the *RAW*-caused pipeline stall. In particular, for the *ntt*-related kernels, the proportion of pipeline stall caused by *RAW* is 9.2% in average. Furthermore, we investigate the instruction distance in one RAW issue. As shown in Fig. 10, 73.7% of RAW issues have 1-instruction distance, while the instruction distance for 16.6% of RAW issues is 2. In CPU, the *forwarding technique* is used for reducing the affection of RAW issues, which stores the results of former instruction in the pipeline and allows to directly use them before their writeback are finished. However, since the core number of GPU is much larger than CPU, directly utilizing a similar forwarding technique in GPU is infeasible due to the

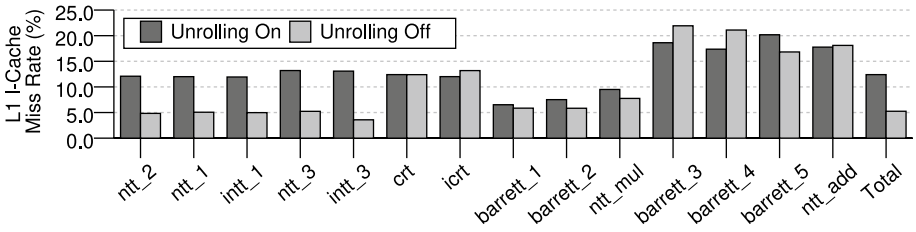



**Fig. 7** The GPU L1 I-Cache miss rate of different FHE kernels

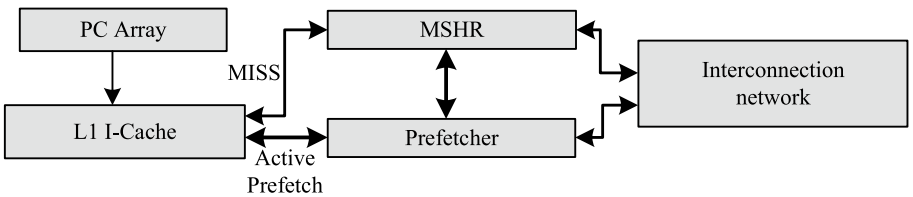



**Fig. 8** The prefetching scheme used in L1 I-Cache

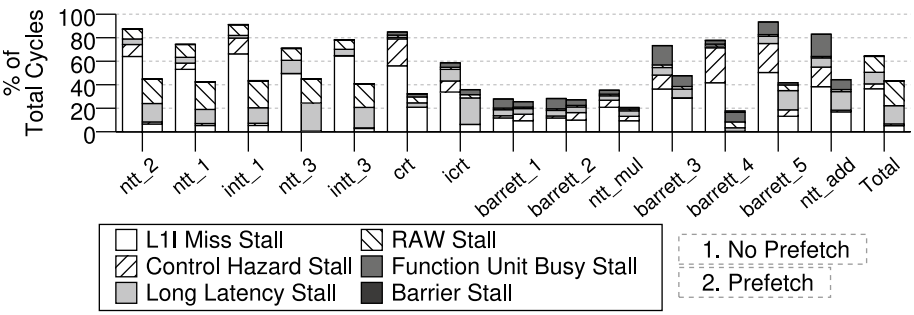



**Fig. 9** The impact of L1 I-Cache prefetching on the pipeline stall

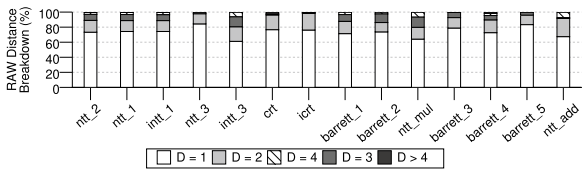



**Fig. 10** The RAW distance breakdown

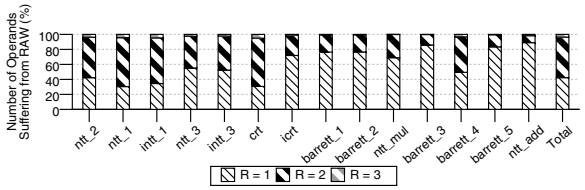



**Fig. 11** The number of operands which suffer from the RAW issue in one instruction

hardware overhead. MoRF (Gilani et al. 2013) allows each SP to hold the most recent three operands for achieving the forwarding, which is based on the extra overhead equal to 55.1% of the register file. However, when we investigate the number of operands that suffer RAW issues in one instruction, we find that buffer provided by MoRF is far from enough. As shown in Fig. 11, for the whole FHE workflow, over half of the instructions contain two or more operands which have to wait for the write-back (57.6%), and this proportion can be up to 69.6% in `ntt_1`. Therefore, we can infer that MoRF (Gilani et al. 2013) is ineffective for reducing RAW stalls in FHE, since the buffer for the recent operands may be filled quickly.

### Challenges and opportunities

Based on the above observations, we conclude our opportunities for boosting the FHE execution on GPU as follow:

- Turn off the unrolling technique in the compilation to reduce the instruction number.
- Using next-line prefetching technique for L1 I-Cache to reduce the stalls.
- Applying shared buffer in each SM to support forwarding to address RAW issue.

Some prior works (Esfeden et al. 2020; Kloosterman et al. 2017; Gebhart et al. 2011) have designed register-level cache to temporarily store the recently used register values. However, these works have some limitations for the RAW problem studied in this paper. Firstly, registers contain temporary values that are frequently renamed, which reduces temporal locality in the cache (Sadrosadati et al. 2020). Second, register-level cache stores all register accesses, including both read and write operands. Since one instruction usually requires two source registers and one destination register, such design results in frequent replacement and is not effective in solving the RAW issue in FHE. Third, the register-level caches are usually built inside the operand collector, whose capacity is limited and cannot avoid long latency caused by *allocate_cu*, *allocate_reads*, and *dispatch_ready_cu* operations. In contrast, our design will uses the real register name as the access index to avoid the negative impact of register renaming. In addition, our proposed structure works outside the operand collector for fast accessing.

To further reduce the hardware overhead of the shared buffer, here we investigate the impact of the Texture-Cache (T-Cache) capacity on the FHE workflow. We run the simulation with the configurations described in "Backup forwarding buffer" section with various T-Cache capacity and plot the breakdown of execution time and pipeline stall time in Figs. 12 and 13 respectively. Based on the figures, one can observe that, the capacity of T-Cache has a very limited impact on the execution time (up to 8.1%). Besides, for all FHE kernels, a smaller T-Cache capacity will not significantly increase the stall time. That is, the T-cache is under-utilized in GPUs during the FHE execution. Therefore, we can use the partition of T-Cache to work as the victim cache of the forwarding table. Such design will improve the effectiveness of the forwarding table, while significantly reducing its hardware overhead.

Note that, the issues of unrolling and L1 I-Cache stall can be well-resolved by existing techniques and are not the target of this paper. Therefore, in the next section, we will propose a pure hardware solution to significantly

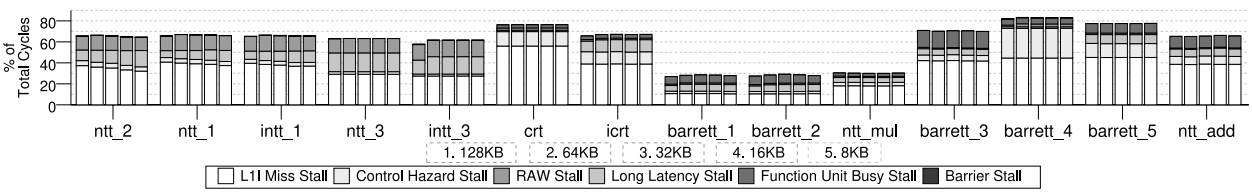



**Fig. 12** The impact of Texture-Cache capacity on the pipeline stall of different kernels

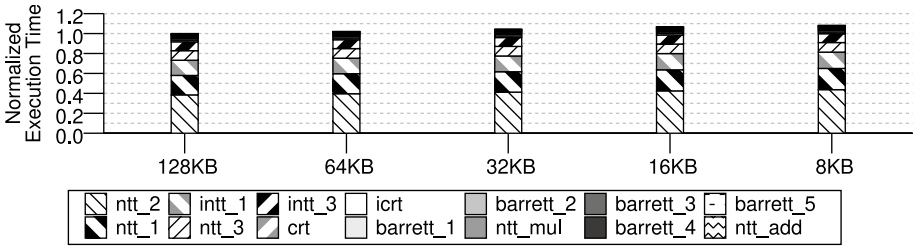



**Fig. 13** The impact of Texture-Cache capacity on the execution time of the whole FHE workflow

reduce the RAW stalls with minor overhead, which is named *MemoPad*.

## MemoPad

Based on the analysis and observation made in "Micro-architecture level analysis for running FHE on GPU" section, we propose the *MemoPad*, which consists of the *Global Forwarding Table (GFT)* and the *Backup Forwarding Buffer (BFB)*. The GFT buffers the value before its writeback is finished and conducts fast access for the subsequent dependent instructions. To reduce the hardware overhead, we also explore to use the idle capacity of the T-Cache to serve as the victim buffer of the proposed GFT, which is called BFB. As a result of its fast operand supplying, GFT significantly reduces the RAW stalls and, together with the L1 I-Cache prefetching technique, boosting the FHE on GPU.

### GFT overview

As shown in Fig. 14, GFT lies beside the scoreboard and is shared by all the warp schedulers in the SM. It buffers the value before it is written back to the Register File. In this way, the following instruction can directly get the required value from the GFT, avoiding the RAW stalls. GFT requires additional modification to the ALU, the scoreboard, and the local memory as follow:

- Upon every warp writeback operation, the ALU sends to GFT a *Warp Instruction Insert Request*, which includes the register information and the to-be-written value (for detailed information, see "Global forwarding table" section).
- When the I-Buffer sends the decoded instruction to the warp scheduler (ⓐ), the warp scheduler checks whether the required register ID is in the scoreboard (ⓑ). If the required register ID is in the scoreboard, the scoreboard sends a Warp Instruction Lookup Request (ⓒ), and then issues the instruction if the GFT contains the required value (ⓔ), or issues a RAW stall signal if the GFT lookup request returns miss; otherwise, the instruction will be directly issued to the next pipeline stage (ⓓ).
- When a writeback is finished, the local memory of the SM sends a message to the GFT to invalidate the corresponding entry.

To further reduce the overhead of GFT, we also explore to utilize the idle capacity in the T-Cache as the victim buffer of the GFT. In this way, the ALU sends the *Warp Instruction Insert Request* to BFB when the GFT is full. When a GFT entry is invalidated, the GFT fetches a new entry from BFB according to the FIFO policy. When a writeback operation is finished, the corresponding entry will be removed, no matter whether it is in the GFT or BFB.

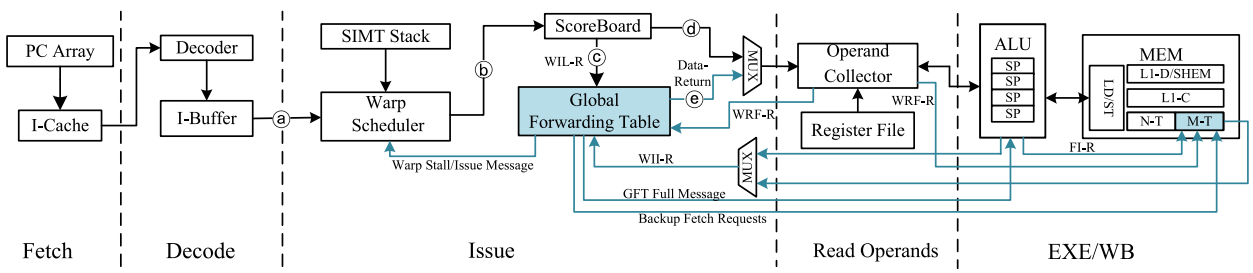



**Fig. 14** A high-level view of *GFT* and its relationship to the other components in the *SM*

## Global forwarding table

### *GFT structure*

As shown in Fig. 15, the GFT is a way-associative hardware structure similar to caches structure, which is managed in a set-associated manner and its storage is divided into the separated tag array and data array. The tag array stores the warp ID, the register ID and the valid identifier, while the data array holds the corresponding values.

### *GFT entry format*

Each GFT entry consists of one tag array entry and one data array entry. It stores the warp ID and the values of all registers with the same register ID in all SPs. As shown in Fig. 15, each GFT entry contains the following fields:

- *Valid (1 bit)*, which indicates the validation of the entry.
- *WID-H (3 bits)*, which records the higher bits of the warp ID. Since the lower bits of the warp ID are used as the set index, the width of the WID-H field varies as the set number changes. In this work, the GFT set number is 4. Therefore, the width of the WID-H field is 3 bits.
- *RID (16 bits)*, which records the register ID.
- *Data (1024 bits)*, which stores all register values with the same register ID in the same SIMD unit. In our paper, each SIMD unit contains 32 SPs, and the capacity of one register is 128 Bytes.

### *Inputs and outputs*

As shown in Fig. 14, the GFT has three types of inputs (i.e., *Warp Instruction Lookup Requests*, *Warp Instruction Insert Requests* and *Warp Instruction Finish Requests*) and four types of output (i.e., *Data Return*, *Warp Stall/Issue Message*, *GFT Full Message* and *Backup Fetch Requests*). The functions of these inputs and outputs are summarized as follow:

- When there is a hit entry in the scoreboard, the scoreboard sends an *Warp Instruction Lookup Request* to the GFT. The request contains the ID of the warp and all required registers. For every *Warp Instruction Lookup Request*, if there is no hit GFT entry for all required registers, the GFT output a *Warp Stall Message* to the Warp Scheduler to issue a pipeline stall; otherwise, the GFT outputs all required data to the Operand Collector as the *Data Return* and informs the Warp Scheduler to issue the instruction with a *Warp Stall/Issue Message*.

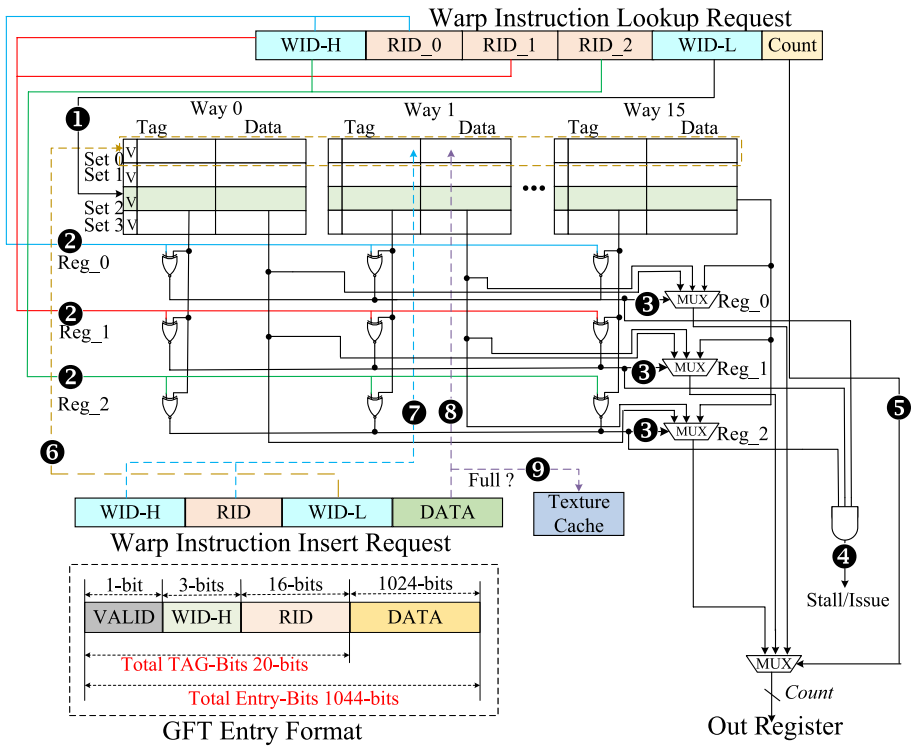



**Fig. 15** The search and insertion in GFT

- A *Warp Instruction Insert Request* can be sent from either the ALU or BFB. For every *Warp Instruction Insert Request*, if there is a valid GFT entry, the information in the request will be stored in it; otherwise, the request will be directly dropped. Note that, if the dropped *Warp Instruction Insert Request* is from the ALU, the GFT returns a *GFT Full Message* to the ALU.
- When a writeback is finished in the local memory, the memory module sends a *Warp Instruction Finish Request* to the GFT. For every *Warp Instruction Finish Request*, the GFT invalidates the corresponding entry if there is a hit GFT entry. If the GFT has a victim buffer in the T-Cache, it then sends a *Backup Fetch Request* to BFB to get a new record.

#### *Warp instruction insert*

Warp Instruction Insert refers to the procedure in which the GFT records the to-be-writeback register values from the ALU. Besides the register value, the Warp Instruction Insert request from the ALU also contains information about the warp and the register. Based on such information, the GFT conducts the following operations:

- The GFT uses the lower 2 bits of the warp ID to select the proper set (❻).
- Then, the GFT checks whether there is any valid entry in the set. If there is a valid entry, the GFT records the higher 3 bits of the warp ID, the register ID and the data in the entry (❼ and ❽); otherwise, the information will be recorded in BFB (❾). Note that, to simplify the hardware implementation, if BFB is full or there is no victim buffer for the GFT, the information of the request will be directly dropped.

#### *Warp instruction lookup*

On a Warp Instruction Lookup (WIL) request, the GFT first checks whether there are any hit entries. If there is one or more hit entry, the value will be successively output to the Out Register (❷ and ❸). Note that, the number of register ID that one WIL Request contains is up to three. Therefore, the GFT generates the signal to issue the instruction only when the entries for all requested register ID are hit; otherwise, the GFT informs the pipeline to stall until all requested operands are ready (❹). Besides, the WIL request also contains the number of the requested registers, which will be used to check if all values in the hit entry are already output (❺).

### Backup forwarding buffer

#### *BFB structure*

As shown in Fig. 16, the BFB is based on the modification to the T-Cache, which divides the original storage array into two parts. For example, in our proposed structure, the successive 64 ways of the second half (Way-64 to Way-127) are utilized by BFB. In this way, the normal requests to the T-Cache only access the Way-0 to Way-63, while the other half only serves the requests to the BFB. Note that, to reduce the access latency of BFB, we add a pointer buffer to the BFB, which is called then *Head-Tail Buffer (HTB)*. The entry number of HTB is equal to the set number of BFB storage array. Each HTB entry stores the head pointer and the tail pointer, which indicate the index of the first and the last valid entry in the corresponding BFB set. For a write access to the BFB storage array, the corresponding tail pointer of the target set will be increased by 1, then the write request will be fulfilled according to the tail pointer; on the other hand, for a read access to the BFB storage array, BFB uses the head pointer of the target set to find the entry for the response, the head pointer of the target set will be increased by 1. Once the head or tail pointer reaches the last way, it will return to the first way on the next request to it. In this way, the access latency to the BFB can be significantly reduced.

#### *Entry format*

The BFB entry format is the same as the GFT entry. Note that, since the width of the T-Cache tag entry (24 bits) is larger than the GFT tag entry (20 bits), we only use part of the T-Cache tag entry to store the GFT tag information. In our work, the capacity waste in the T-Cache is 0.2%.

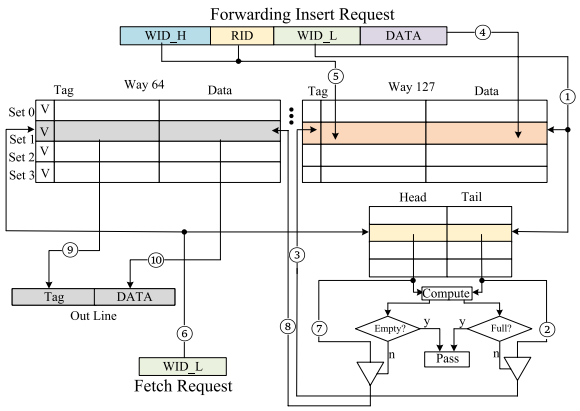



**Fig. 16** The insertion in BFB

#### *Input and output*

As shown in Fig. 14, the BFB has three types of input (i.e., *Forwarding Insert Requests*, *Backup Fetch Requests* and *Warp Instruction Finish Requests*) and one output (*Warp Instruction Insert Request*). The *Forwarding Insert Request* is from the ALU and informs the BFB to record the information in a valid entry. If there is no valid entry, the *Forwarding Insert Request* will be dropped. For every *Backup Fetch Request* from the GFT, the BFB selects the proper entry and returns it to the GFT by issuing a *Warp Instruction Insert Request*. The *Warp Instruction Finish Request* from the local memory module indicates that the writeback is finished and the BFB will remove the corresponding entry.

#### *Forwarding insert*

Forwarding Insert refers to the procedure in which the BFB records the to-be-writeback values from the ALU when the GFT is full. On a Forwarding Insert request, the BFB firstly checks the corresponding HTB entry of the target set according to the Warp ID contained in the request(①). Once the head and tail pointer is the same, it indicates that the BFB is full and the Forwarding Insert request information will be dropped; otherwise, the BFB storage array obtains the entry index in the target set (② and ③) and serves the Forwarding Insert request as a write access(④ and ⑤).

#### *Backup fetch*

Backup Fetch refers to the procedure in which the GFT requests an entry from the BFB. The request contains only the low 2-bit of the warp ID. On a Backup Fetch request, the BFB first access the corresponding HTB entry according to the information in the request(⑥), and then checks if the head pointer is equal to the tail pointer. Once the two pointers are the same, it indicates that the corresponding set is empty, and the BFB will return an invalid message to the GFT; otherwise, the BFB serves the request as a read access (⑦ and ⑧). Then the BFB returns the entry according to the head pointer and set the corresponding entry in the storage array as invalid (⑨ and ⑩). Since the GFT tag width is different from the cache, the request will drop the lowest 4-bit of cache tag.

## Methodology

We implement our proposed structure based on the GPGPUSim 4.0 (Khairy et al. 2020), which provides a cycle-accurate simulation for modern GPU architecture. The configuration is the same as the baseline shown in Table 3. We modify the simulator to implement a GFT in each SM with a 1044-bit entry size, 4 sets, and 16-way associativity. Such a GFT requires about 8.16KB of storage, which is only 6.4% of the storage required by the Register File. For the L1 T-Cache, we modify its circuit and utilize half of the storage array as the *BFB*. The BFB partition has 4 sets and 16-way associativity. Besides, we employ the RTL method to conduct the post-synthesis for the proposed GFT and the BFB structure and then use the Synopsis Design Compiler (v2016) to measure their access latency, the circuit area, and the power consumption. Based on the synthesis with TSMC 7*nm* technology library, the access latency of the GFT and the BFB are 2 cycles in respective, while the power consumption of GFT is 0.0578*W*, which is only 0.02% of the GPU peak power.

Table 4 shows the simulated schemes in our experiments. The *Baseline* refers to the scheme without any optimization for FHE. The *Prefetch* refers to the scheme with the next-line prefetching technique (Smith 1982) in L1 I-Cache. For our simulated prefetching schemes, the prefetching depth is 1 and the

**Table 3** Simulation configurations

| Module | Configuration |
|---|---|
| GPU | |
| SM | 1.132 GHz, 16 on GPU, 32 warps/SM |
| Register File | 256KB/SM (65536 Reg), 4-cycle (Sadrosadati et al. 2020) |
| L1 I-Cache | 8 KB, 2-way, 20-cycle access delay |
| L1 D-Cache | 128 KB, 256-way, 20-cycle access delay |
| Texture Cache | 64 KB, 128-way, 20-cycle access delay |
| L2 Cache | 6MB, 24-way, 160-cycle access delay |
| DRAM | 16 GB, 260-cycle access delay |
| ScoreBoard | 32-entry |
| Warp scheduler | 4 per SM, GTO scheduling police |
| GFT | |
| GFT | 16-way, 4-set, 1044 bits per entry<br>2-cycle access delay |
| BFB | 4-set, 64-way, 2-cycle access delay |

**Table 4** Simulated schemes

| Scheme | Description |
|---|---|
| *Baseline* | GPU without optimization for FHE |
| *Prefetch* | Add prefetching technique (Smith 1982) for The L1 I-Cache of the *Baseline*<br>Prefetching depth is 1 |
| *MoRF Only* | Add MoRF to the Baseline |
| *MemoPad Only* | Add MemoPad to the Baseline |
| *MoRF+Prefetch* | Add MoRF to the *Prefetch* |
| *GFT+Prefetch* | Add single GFT to the *Prefetch* |
| *MemoPad+Prefetch* | Add MemoPad to the *Prefetch* |

L1 I-Cache also issues the active prefetching request when there is free entry in the I-Buffer (as shown in Fig. 8). The *MoRF Only* refers to the scheme that only adds the forwarding structure as introduced in Ref. Gilani et al. (2013) to the *Baseline* scheme. The *MemoPad Only* refers only adding the *MemoPad* structure to the scheme. Besides the above individual solutions, we also simulate the solutions integrated with the prefetching technique (refered as the *Prefetch+MoRF*, *Prefetch+GFT* and *Prefetch+MemoPad*).

To evaluate our proposed structures, we use the ATV-FHE kernels and the FHE workflow from the cuHE (Dai and Sunar 2015) library (as shown in Table 2) as the benchmark. In the experiments, we assume that the RAW-1 issues are eliminated with instruction reordering techniques (Rawat et al. 2018) for all our proposed schemes. With each simulated scheme, we collect the data for each kernel and the whole workflow. Note that, since our proposed MemoPad does not require the modification in the FHE applications, it has no impact on the security (Fig. 17).

## Results

In order to evaluate the effectiveness of our proposed MemoPad, we compare our proposed scheme to a series of schemes. The evaluation metrics include performance, pipeline stall time and the hardware overhead. Then we present the efficiency of the BFB. Finally, we provide the sensitivity study to the different design options of MemoPad.

### Performance

We use the total execution time to denote the performance, which includes the compute time and the pipeline stall time that cannot be overlapped by the warp context switching. The results are as shown in Fig. 18, and we can make the following conclusions:

- *Individual schemes comparison* Comparing to the baseline, the individual *MemoPad Only* improves the performance of the whole workflow by 21.7% and the performance of various kernels by 8.8% as the average. This is because the *MemoPad Only* reduces the pipeline stall caused by RAW issues. However, the total performance improvement of *MemoPad Only* is 6.4% lower than the *Prefetch* scheme, since the proportion of the pipeline stall time caused by RAW stall is lower than the L1 I-Cache miss. When compared to the *MoRF Only*, the *MemoPad* improves the performance of total workflow by 16.4%, since GFT provides the shared buffer for all SIMD units and enhances the utilization.
- *Prefetch-integrated schemes comparison* When we consider the integration with the prefetching technique, all schemes gain the performance improvement. For example, for the overall workflow, the *Prefetch+GFT* performs a significant improvement to the baseline by 47.5% and 27.5% higher than the *Prefetch* standalone scheme. This is because the *Prefetch+GFT* scheme reduces most of the pipeline stall, while the GFT helps to remove the negative impact on the RAW stall caused by L1 I-Cache prefetching. Besides, the *Prefetch+MemoPad* scheme

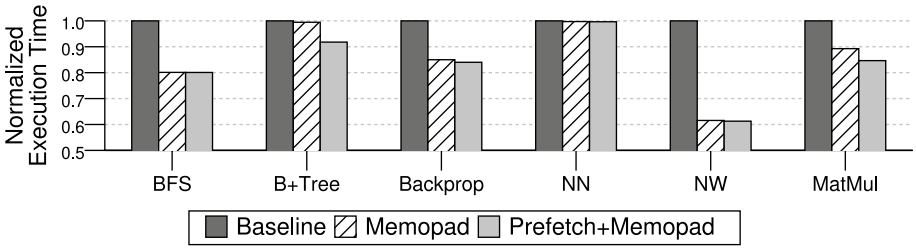



**Fig. 17** The performance comparison for normal kernels by using MemPad

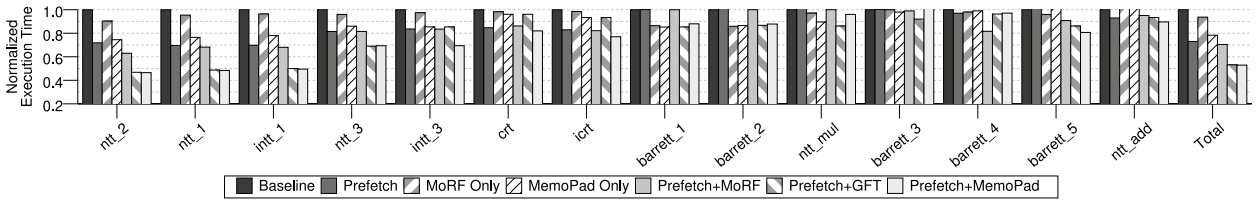



**Fig. 18** Normalized execution time comparison

provides 5.6% further performance improvement to the *Prefetch+GFT* scheme.

Besides, we also evaluate how *MemoPad* affects the performance of normal workloads with the applications from the Rodinia benchmark suite (Che et al. 2009). As shown in Fig. 17, for all kinds of applications, *MemoPad* has no negative impact on the performance and achieves obvious speedup in some of the applications (i.e., `BFS`, `Backprop` and `NW`).

#### Stall time breakdown

Table 5 presents the overall pipeline stall time breakdown in the GPU with different schemes. When comparing to the *Baseline*, the individual *MemoPad* scheme significantly reduces the RAW stall by 86.7%, while the L1 I-Cache miss stall proportion minorly increases by 7.4%. This is because the *MemoPad* reduces the RAW stall and allows more frequently accesses to the L1 I-Cache. When comparing the *Prefetch* scheme with the individual *MemoPad* scheme, we can see that though the *Prefetch* scheme reduces the L1 I-Cache Miss Stall by 74.9%, it also increases the RAW stall by 1.1×. Because the *Prefetch* scheme significantly increases the issued instructions and leads to more possible RAW stalls. Overall, the *Prefetch+MemoPad* scheme performs 88.2% less total stall time than the baseline, while the reduction of L1 I-Cache Miss Stall and RAW Stall are 95.9% and 87.1%.

#### BFB efficiency

The efficiency of the BFB indicates the proportion of the BFB entry be fetched to the GFT and accessed. Originally, as we discussed in "Backup forwarding buffer" section, the GFT fetches an entry from the BFB when it has free space. When a writeback request is finished, its corresponding GFT or BFB entry will be invalided. Therefore, the higher efficiency indicates that the BFB provides more support to the performance increment.

**Table 6** The efficiency of the BFB

| Kernel | Efficiency (%) | Kernel | Efficiency (%) |
|---|---|---|---|
| ntt_2 | 62.1 | barrett_1 | 43.6 |
| ntt_1 | 53.8 | barrett_2 | 43.1 |
| intt_1 | 59.7 | ntt_mul | 30.1 |
| ntt_3 | 56.1 | barrett_3 | 42.9 |
| intt_3 | 53.4 | barrett_4 | 28.9 |
| crt | 30.9 | barrett_5 | 22.6 |
| icrt | 43.1 | ntt_add | 43.7 |

As shown in Table 6, for various kernels, the BFB efficiency is 43.9% in average and up to 62.1% for *nnt_2*. The *barrett_5* has the lowest BFB efficiency (22.6%), indicating most of the entries will not be used before it is removed.

#### Sensitivity study to GFT capacity

We further evaluate the overall performance of FHE with different GFT capacity configurations. As shown in Table 7, the `G-N` configuration represents the GFT with N-way associativity. A larger GFT can reduce the entry fetching from the BFB and increase the possibility of directly issuing the instructions. For the extremely performance improvement, a larger GFT is preferred, at the expense of higher overhead.

As shown in Fig. 19 and Table 7, when the capacity of GFT increases from 64-entry to 256-entry, the overall performance increases by 7.1%, while the hardware overhead increases by 3.8×. On the other hand, as the GFT capacity decreases from 64-entry to 8-entry, the overall performance decreases by 31.3%, but the hardware overhead also reduces by 83.5%. Besides, the Fig. 19 also illustrates that the GFT capacity has the most significant impact on the kernel *ntt_1*.

**Table 5** Overall stall time breakdown with different schemes

| | Baseline | Prefetch | MoRF only | MemoPad only | MoRF + Prefetch | GFT + Prefetch | MemoPad + Prefetch |
|---|---|---|---|---|---|---|---|
| L1-I miss stall (million cycles) | 236.6 | 24.6 | 241.9 | 95.3 | 22.5 | 10.1 | 9.7 |
| Control hazard stall (million cycles) | 26.8 | 7.1 | 28.3 | 22.7 | 6.1 | 6.3 | 6.2 |
| Long latency stall (million cycles) | 63.8 | 73.9 | 67.4 | 17.5 | 84.2 | 20.3 | 20.1 |
| RAW stall (million cycles) | 87.3 | 99.5 | 66.9 | 11.6 | 81.9 | 13.6 | 11.2 |
| Function unit busy (million cycles) | 1.2 | 0.7 | 0.9 | 1.0 | 1.5 | 1.5 | 1.7 |
| Barrier stall (million cycles) | 1.0 | 0.5 | 0.7 | 0.3 | 0.5 | 0.2 | 0.3 |
| Total stall time (million cycles) | 416.7 | 206.4 | 406.2 | 148.4 | 196.7 | 51.9 | 49.1 |
| Stall time proportion | 64.4% | 43.3% | 66.9% | 29.3% | 43.2% | 15.1% | 14.3% |

**Table 7** The overhead for the GFT capacity

| | G-64 | G-32 | G-16 | G-8 | G-4 | G-2 |
|---|---|---|---|---|---|---|
| Area ($mm^2$) | 16.41 | 8.28 | 4.21 | 2.27 | 1.22 | 0.71 |
| OVHD (%) | 1.99 | 1.00 | 0.51 | 0.27 | 0.15 | 0.09 |

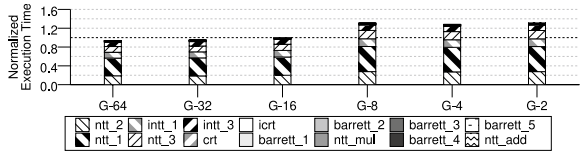



**Fig. 19** The sensitivity study to the GFT capacity

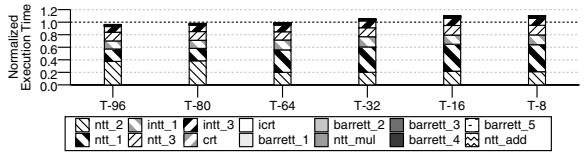



**Fig. 20** The sensitivity study to the BFB capacity

### Sensitivity study to BFB capacity

We further analyze the impact of the BFB capacity. As we introduced in "Backup forwarding buffer" section, a larger BFB helps to store more entries for GFT and increases the possibility of performance improvement. However, the increment of the BFB capacity also leaves less capacity for the original T-Cache, which may leads to the negative impact on the overall stall time and performance. As shown in Fig. 20, the `T-N` configuration refers to the BFB with N-way associativity. When we increase the associativity of the BFB from 64 to 96, the overall performance is improved by 4.8%. On the other hand, when the associativity of the BFB decrease from 64 to 8, the overall performance decreases by 10.1%.

Figure 21 presents the impact of the BFB capacity on the pipeline stall. As shown in the figure, the stall time of most kernels increases as the capacity of the BFB decreases. This indicates that the BFB capacity provides much more significant impact on the RAW stall than the T-Cache capacity.

## Related work

As an emerging hot topic, the Homomorphic Encryption suffers from the significant performance gap and cannot be wide adopted. In this section, we briefly summarize the previous efforts for solving this problem.

*Software optimizations for CPU and GPU* For the performance issue, the most straightforward approaches are to parallelize the workload and utilize the massive computation capability of CPU and GPU. For example, Intel HEXL was proposed to accelerate the NTT and modular multiplication with Intel Advanced Vector Extensions 512 (Intel AVX512) instruction set (Boemer et al. 2021), which can obtain convenience in the process of utilizing practical application by integrating with SEAL (Chen et al. 2017) and TenSEAL (Benaissa et al. 2021). Wang et al. leverage Strassen's FFT algorithm and Barrett modular on CPU to accelerate the large number multiplication and modular reduction, which finally achieve averagely 7.3× speedup (Wang et al. 2012). On the other hand, the scheduling optimization is also considered as a kind of effective solution for boosting the FHE. Wonkyung et al. use the kernel fusion and reordering of primary functions to reduce the memory accesses and the kernel launch time, which improves the overall performance by 13.8% (Jung et al. 2021). Sangpyo et al. achieve the 4.2× speedup for NTT operations on GPU by optimizing the batching, the register-based high-radix, and the shared memory usage (Kim et al. 2020). Besides, (Morshed et al. 2020) utilize the bit-merge compound gates to improve the FHE computation parallelism and achieve a 14.5× speedup.

*Hardware accelerators and HW-SW co-design* A series of hardware accelerators have been proposed for improving the performance of FHE. Most of this kind of solutions focus on boosting the key kernels. For example, Erdinc et al. design a large NTT-based multiplier to compute large degree polynomial multiplications using the Cooley-Tukey FFT technique (Öztürk et al. 2016).

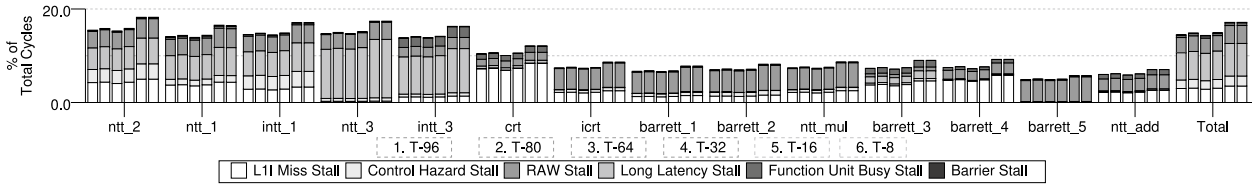



**Fig. 21** The impact of BFB capacity on the stall time

David et al. focus on the CRT/ICRT kernels and propose a computing accelerator based on FPGA to improve the computational practicability of encrypted data, which reduces the computational bottleneck of lattice encryption primitives supporting HE schemes and achieves 4.2× speedup (Cousins et al. 2016). F1 utilizes the wide-vector processor to accelerate the polynomial-related kernels while reducing the data movement for the FHE procedure and remarkable gains up to 17,000× speedup (Samardzic et al. 2021). Besides, some system-level solutions are also proposed in the approach of software-hardware co-design in the recent years. For instance, Cheetah achieves 79× speedup for FHE by using the ASIC accelerators for boosting the key kernels of FHE and a special-designed workload management system to optimize the hardware utilization. In this way, it supports an inference system with privacy protection feature (Reagen et al. 2021). Reis et al. (2020) design a CiM hardware consisting of customized peripherals, which reduces latency and energy associated with data transfer in data-intensive HE applications and offers 2.2× speedup with 88.1× energy savings compared to a state-of-the-art FPGA-based accelerator.

## Discussion

### Discussion of different schemes

While this work uses ATV-FHE (DHS) as the primary evaluation target, the underlying microarchitectural issues we identify are widely applicable to other mainstream FHE schemes such as BFV (Bajard et al. 2017), CKKS (Cheon et al. 2017), and even TFHE (Chillotti et al. 2020). Fundamentally, these schemes all rely on lattice-based cryptography and are composed of a large number of fine-grained, arithmetic-intensive primitives such as modular multiplications, NTT/INTT, and key-switching operations. These operations induce highly irregular and instruction-divergent execution patterns, leading to severe instruction cache pressure and frequent instruction cache misses. As such, the instruction starvation and RAW-induced pipeline stalls observed in our work are not exclusive to ATV-FHE but are inherent to lattice-based FHE computation in general.

Moreover, GPU-based acceleration of FHE workloads, regardless of the specific scheme, commonly suffers from inefficient data movement and poor memory hierarchy utilization. In particular, we observe that the texture cache (T-cache) remains largely under-utilized during FHE execution due to the lack of spatial locality and irregular access patterns typical of cryptographic arithmetic. These issues align with those seen in our profiling of ATV-FHE and point to a broader challenge in adapting GPU architectures for FHE workloads. Given that our proposed architectural optimizations—including RAW stall mitigation, instruction prefetch enhancements, and texture cache repurposing—target these fundamental hardware inefficiencies, we believe the techniques can be naturally extended to benefit other lattice-based FHE schemes as well. Future work will explore this generalization in more depth by integrating BFV and CKKS workloads into our evaluation.

### Discussion of recent GPU optimization

In recent years, several works have explored software-level optimizations on existing GPU architectures to improve the execution efficiency of FHE workloads. TensorFHE attempts to transform NTT computations into matrix multiplications to alleviate scheduling conflicts caused by RAW dependencies (Fan et al. 2023). However, it does not fundamentally resolve the write-back latency issue and leaves the problem of poor instruction locality unaddressed. WarpDrive further proposes a hybrid approach combining matrix multiplication and butterfly operations to reduce in-place writes and mitigate RAW bottlenecks (Fan et al. 2025). It also leverages PTX-level optimizations to parallelize TCU and CUDA Core execution, thereby improving pipeline utilization. Neo aims to offload more computation to TCUs to enhance overall GPU resource utilization (Jiao et al. 2025), while Cheddar adopts a hardware-software co-design strategy to accelerate CKKS workloads (Kim et al. 2024).

Nonetheless, these approaches primarily focus on software-level scheduling and mapping, leaving fundamental architectural issues—such as RAW hazards, instruction cache inefficiencies, and low texture cache utilization—largely unresolved. In contrast, the hardware-level optimizations proposed in this work are orthogonal to these efforts and can further enhance their effectiveness.

## Conclusion

In this paper, we provide the first micro-architectural level analysis for the execution of FHE on GPU. According to our experiment results, the FHE performance suffers from the low hardware utilization, which is fundamentally caused by the high L1 I-Cache miss rate and the frequent RAW issues. To solve these problems, we explore the optimized technique combination for reducing the L1 I-Cache misses and propose a pure hardware solution with minor overhead for reducing the RAW issues. Our proposed hardware structure, which is named *MemoPad*, adds a shared cache-like forwarding buffer in each SM and uses the partition of the T-Cache as the victim buffer. With *MemoPad*, the instructions can directly use the operands before its writeback is finished and the RAW issues are significantly reduced. In the evaluations, we quantatively compare the propsed *MemoPad* with the previous related work and explore the impacts of the key parameters.

We believe that the analysis in this paper can inspire the future work of algorithm and schedule optimization for running FHE on GPU. Furthermore, the experience and identified issues can also help to build better FHE accelerators.

**Acknowledgements**
Not applicable.

**Author contributions**
Shengyun Fan drafted the manuscript. Xianglong Deng contributed to its revision. Mingzhe Zhang, Weizhi Xu, and Xulong Tang supervised the work.

**Funding**
This research was supported by the National Key R&D Program of China (Grant No. 2023YFB4503201)
and the Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No. XDB44030200).

**Data availability**
Data is available within the article.

## Declarations

**Competing interests**
The authors declare that they have no conflict of interest.

Received: 22 April 2025 Accepted: 10 September 2025
Published online: 19 March 2026

**References**

Bajard J-C, Eynard J, Hasan MA, Zucca V (2017) A full RNS variant of FV like somewhat homomorphic encryption schemes. In: Selected areas in cryptography—SAC 2016: 23rd international conference, St. John's, NL, Canada, August 10–12, 2016, Revised Selected Papers, pp 423–442. Springer

Benaissa A, Retiat B, Cebere B, Belfedhal AE (2021) Tenseal: a library for encrypted tensor operations using homomorphic encryption. arXiv preprint arXiv:2104.03152

Boemer F, Kim S, Seifu G, Souza FD, Gopal V (2021) Intel hexl: accelerating homomorphic encryption with intel avx512-ifma52. arXiv preprint arXiv:2103.16400

Brutzkus A, Gilad-Bachrach R, Elisha O (2019) Low latency privacy preserving inference. In: International conference on machine learning, pp 812–821. PMLR

Che S, Boyer M, Meng J, Tarjan D, Sheaffer JW, Lee S-H, Skadron K (2009) Rodinia: a benchmark suite for heterogeneous computing. In: 2009 IEEE international symposium on workload characterization (IISWC), pp 44–54. IEEE

Chen H, Laine K, Player R(2017) Simple encrypted arithmetic library-seal v2. 1. In: International conference on financial cryptography and data security, pp 3–18. Springer

Cheon JH, Kim A, Kim M, Song Y (2017) Homomorphic encryption for arithmetic of approximate numbers. In: International conference on the theory and application of cryptology and information security, pp 409–437. Springer

Chillotti I, Gama N, Georgieva M, Izabachène M (2020) TFHE: fast fully homomorphic encryption over the torus. J Cryptol 33(1):34–91

Cousins DB, Rohloff K, Sumorok D (2016) Designing an FPGA-accelerated homomorphic encryption co-processor. IEEE Trans Emerg Top Comput 5(2):193–206

Dai W, Sunar B (2015) cuhe: a homomorphic encryption accelerator library. In: International conference on cryptography and information security in the Balkans, pp 169–186. Springer

Dathathri R, Saarikivi O, Chen H, Laine K, Lauter K, Maleki S, Musuvathi M, Mytkowicz T (2019) Chet: an optimizing compiler for fully-homomorphic neural-network inferencing. In: Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation, pp 142–156

Dathathri R, Kostova B, Saarikivi O, Dai W, Laine K, Musuvathi M (2020) EVA: an encrypted vector arithmetic language and compiler for efficient homomorphic computation. In: Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation, pp 546–561

Doroz Y, Hu Y, Sunar B (2014) Homomorphic AES evaluation using NTRU. Cryptology ePrint Archive

Esfeden HA, Abdolrashidi A, Rahman S, Wong D, Abu-Ghazaleh N (2020) BOW: breathing operand windows to exploit bypassing in GPUs. In: 2020 53rd annual IEEE/ACM international symposium on microarchitecture (MICRO), pp 996–1008. IEEE

Fan J, Vercauteren F (2012) Somewhat practical fully homomorphic encryption. IACR Cryptol ePrint Arch 2012:144

Fan S, Wang Z, Xu W, Hou R, Meng D, Zhang M (2023) Tensorfhe: achieving practical computation on encrypted data using GPGPU. In: 2023 IEEE international symposium on high-performance computer architecture (HPCA), pp 922–934. IEEE

Fan G, Zhang M, Zheng F, Fan S, Zhou T, Deng X, Tang W, Kong L, Song Y, Yan S (2025) Warpdrive: GPU-based fully homomorphic encryption acceleration leveraging tensor and CUDA cores. In: 2025 IEEE international symposium on high performance computer architecture (HPCA), pp 1187–1200. https://doi.org/10.1109/HPCA61900.2025.00091

Gebhart M, Keckler SW, Dally WJ (2011) A compile-time managed multi-level register file hierarchy. In: 2011 44th Annual IEEE/ACM international symposium on microarchitecture (MICRO), pp 465–476. IEEE

Gentry C (2009) A fully homomorphic encryption scheme. Stanford University

Gilani SZ, Kim NS, Schulte MJ (2013) Exploiting GPU peak-power and performance tradeoffs through reduced effective pipeline latency. In: 2013 46th Annual IEEE/ACM international symposium on microarchitecture (MICRO), pp 74–85. IEEE

Han K, Hong S, Cheon JH, Park D (2019) Logistic regression on homomorphic encrypted data at scale. Proceedings of the AAAI Conference on Artificial Intelligence 33:9466–9471

Jiao D, Deng X, Wang Z, Fan S, Chen Y, Meng D, Hou R, Zhang M (2025) Neo: towards efficient fully homomorphic encryption acceleration using tensor core. In: Proceedings of the 52nd annual international symposium on computer architecture, pp 107–121

Jung W, Kim S, Ahn JH, Cheon JH, Lee Y (2021) Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. IACR Trans Cryptogr Hardw Embedded Syst 2021:508

Khairy M, Shen Z, Aamodt TM, Rogers TG (2020) Accel-sim: an extensible simulation framework for validated GPU modeling. In: 2020 ACM/IEEE 47th annual international symposium on computer architecture (ISCA), pp 473–486. IEEE

Kim J, Choi W, Ahn JH (2024) Cheddar: a swift fully homomorphic encryption library for CUDA GPUs. arXiv preprint arXiv:2407.13055

Kim S, Jung W, Park J, Ahn JH (2020) Accelerating number theoretic transformations for bootstrappable homomorphic encryption on GPUs. In: 2020 IEEE international symposium on workload characterization (IISWC), pp 264–275. IEEE

Kloosterman J, Beaumont J, Jamshidi DA, Bailey J, Mudge T, Mahlke S(2017) Regless: just-in-time operand staging for gpus. In: 2017 50th Annual IEEE/ACM international symposium on microarchitecture (MICRO), pp 151–164 . IEEE

Morshed T, Al Aziz MM, Mohammed N (2020) CPU and GPU accelerated fully homomorphic encryption. In: 2020 IEEE international symposium on hardware oriented security and trust (HOST), pp 142–153 . IEEE

Murthy GS, Ravishankar M, Baskaran MM, Sadayappan P (2010) Optimal loop unrolling for GPGPU programs. In: 2010 IEEE international symposium on parallel & distributed processing (IPDPS), pp 1–11. IEEE

Öztürk E, Doröz Y, Savaş E, Sunar B (2016) A custom accelerator for homomorphic encryption applications. IEEE Trans Comput 66(1):3–16

Rawat PS, Sukumaran-Rajam A, Rountev A, Rastello F, Pouchet L-N, Sadayappan P (2018) Associative instruction reordering to alleviate register pressure. In: SC18: international conference for high performance computing, networking, storage and analysis, pp 590–602. IEEE

Reagen B, Choi W-S, Ko Y, Lee VT, Lee H-HS, Wei G-Y, Brooks D (2021) Cheetah: optimizing and accelerating homomorphic encryption for private inference. In: 2021 IEEE international symposium on high-performance computer architecture (HPCA), pp 26–39. IEEE
Reis D, Takeshita J, Jung T, Niemier M, Hu XS (2020) Computing-in-memory for performance and energy-efficient homomorphic encryption. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 28(11):2300–2313
Rogers TG, O'Connor M, Aamodt TM (2012) Cache-conscious wavefront scheduling. In: 2012 45th Annual IEEE/ACM international symposium on microarchitecture, pp 72–83. IEEE
Sadrosadati M, Mirhosseini A, Hajiabadi A, Ehsani SB, Falahati H, Sarbazi-Azad H, Drumond M, Falsafi B, Ausavarungnirun R, Mutlu O (2020) Enabling high-capacity, latency-tolerant, and highly-concurrent GPU register files via software/hardware cooperation. arXiv preprint arXiv:2010.09330
Samardzic N, Feldmann A, Krastev A, Devadas S, Dreslinski R, Peikert C, Sanchez D (2021) F1: a fast and programmable accelerator for fully homomorphic encryption. In: MICRO-54: 54th annual IEEE/ACM international symposium on microarchitecture, pp 238–252
Smith AJ (1982) Cache memories. ACM Comput Surv 14(3):473–530
Wang W, Hu Y, Chen L, Huang X, Sunar B(2012) Accelerating fully homomorphic encryption using GPU. In: 2012 IEEE conference on high performance extreme computing, pp 1–5. IEEE
Yagisawa M (2015) Fully homomorphic encryption without bootstrapping. IACR Cryptol EPrint Arch 2015:474

## Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.